

SE2026 股票交易系统 交易系统管理业务

【股票交易系统】

——交易系统管理业务

系统设计报告

组长：孙米阳

组员：郝建文 黄中维 杨佳利 陆思越

日期：2026 年 5 月 25

版本：V1.0

目录

1 文档介绍	5
1.1 编写目的	5
1.2 文档范围	5
1.3 读者对象	5
1.4 术语与缩写解释	5
1.5 参考资料	6
2 项目介绍	7
2.1 项目说明	7
2.2 项目背景	7
2.3 需求概述	7
2.3.1 功能需求	7
2.3.2 性能需求	8
2.3.3 安全性需求	8
2.4 条件与限制	8
2.4.1 技术约束	8
2.4.2 外部依赖	8
2.4.3 约束限制	8
3 总体设计	9
3.1 基本设计概念和流程处理	9
3.1.1 设计概念	9
3.1.2 核心流程概述	9
3.2 功能 IPO 图	9
3.3 系统结构	10
3.4 技术介绍	11
3.5 部署图	11
3.6 类图	12
3.6.1 实体类 (Model)	12
3.6.2 服务类 (Service)	13
3.6.3 外部接口类 (Client)	13
3.7 接口设计	13
3.7.1 内部接口	13
3.7.2 外部接口 (ADMIN 对外提供的 API)	14
3.7.3 外部接口 (ADMIN 调用的 TRADE API)	15
4 详细设计	16
4.1 顺序图	16
4.1.1 管理员登录认证流程	16
4.1.2 股票查看流程	17
4.1.3 涨跌停设置流程	18
4.1.4 交易暂停与重启流程	19
4.1.5 权限管理流程	21
4.1.6 审计日志查询流程	22
4.2 执行概念	22
4.2.1 登录认证执行逻辑	22

4.2.2	涨跌停设置执行逻辑	22
4.2.3	交易控制执行逻辑	23
4.2.4	密码管理执行逻辑	23
4.2.5	权限管理执行逻辑	23
5	用户界面	24
5.1	设计原则	24
5.2	界面原型	24
5.2.1	登录界面	24
5.2.2	股票查看主界面	24
5.2.3	涨跌停设置界面	24
5.2.4	交易控制界面	24
5.2.5	权限管理界面	25
5.2.6	审计日志界面	25
5.2.7	密码修改界面	25
5.3	页面流转关系	25
6	数据库设计	26
6.1	概念结构设计	26
6.1.1	实体描述	26
6.1.2	实体间关系	26
6.1.3	E-R 图	27
6.2	逻辑结构设计	27
6.2.1	管理员信息表 (admin_info)	27
6.2.2	操作日志表 (operation_log)	28
6.2.3	权限配置表 (permission_config)	28
6.2.4	登录日志表 (login_log)	28
6.3	物理结构设计	29
6.3.1	存储引擎	29
6.3.2	索引设计	29
6.3.3	备份策略	29
7	运行设计	30
7.1	运行环境	30
7.1.1	服务器环境	30
7.1.2	客户端环境	30
7.2	运行流程	30
7.2.1	系统启动	30
7.2.2	系统运行	30
7.2.3	系统维护	30
7.2.4	系统关闭	31
7.3	运行控制	31
8	系统出错设计	32
8.1	出错信息	32
8.2	补救措施	32
8.2.1	数据备份与恢复	32
8.2.2	故障恢复	32
8.2.3	安全事件响应	32

8.3 系统维护设计	33
------------------	----

1 文档介绍

1.1 编写目的

本文档描述股票交易系统 交易系统管理业务 的系统设计，旨在：

- 1) 在需求规格说明书（SRS）的基础上，将需求转化为系统的体系结构，划分出系统的基本组成模块；
- 2) 定义各模块的内部处理逻辑、数据结构与接口规范，为后续的详细设计与编码实现提供依据；
- 3) 根据跨组接口约定（接口 V2.md），明确本子系统与中央交易系统（TRADE）的交互方式；
- 4) 设计系统的数据库结构、用户界面与运行部署方案。

1.2 文档范围

本文档涵盖股票交易系统 交易系统管理业务 的系统总体设计、模块详细设计、数据库设计、用户界面设计、运行设计与系统出错设计等内容。系统的外部接口设计遵循《股票交易系统五组接口约定 v0.2》规范。

1.3 读者对象

本报告的主要读者为：

- 1) 项目经理及系统分析人员：了解系统总体架构与模块划分；
- 2) 详细设计人员及程序开发人员：依据本文档进行模块实现；
- 3) 测试人员：参照本文档设计测试用例；
- 4) 系统维护人员：了解系统结构与运行方式。

1.4 术语与缩写解释

缩写、术语及符号	解释
SRS	软件需求规格说明书（Software Requirements Specification）。
RBAC	基于角色的访问控制（Role-Based Access Control），通过角色划分管理用户权限。
IPO 模型	输入 -处理 -输出模型（Input-Process-Output），用于描述模块的数据转换逻辑。
DFD	数据流图（Data Flow Diagram），描述系统数据流向与处理的图形化工具。
E-R 图	实体 -关系图（Entity-Relationship Diagram），用于数据库概念设计。
API	应用程序编程接口（Application Programming Interface），定义系统内外部交互的接口规范。
TRADE	中央交易系统（Central Trading System），负责指令接收、撮合、成交、行情推送。
ACCOUNT	账户业务子系统，管理资金账户、证券账户与认证。

INFO	网上信息发布子系统，面向普通/VIP 用户提供行情查询与 K 线展示。
CLIENT	交易客户端，投资者侧的交互界面。
JWT	JSON Web Token，用于无状态身份认证的令牌格式。
SQL 注入	一种代码注入攻击技术，攻击者通过在输入中插入恶意 SQL 语句来操纵数据库。

1.5 参考资料

序号	文档名称	文档编号	版本	发布日期
1	2026 年软件工程基础实验大纲——股票交易系统	SE2026-LAB-OUTLINE	V1.0	2026 年 2 月
2	交易系统管理业务——需求分析报告	SE2026-SRS	V1.0	2026 年 5 月
3	股票交易系统五组接口约定	SE2026-API-V2	v0.2	2026 年 5 月
4	软件工程基础课程讲义	SE2026-LECTURE	V1.0	2026 年 2 月

2 项目介绍

2.1 项目说明

项目名称	股票交易系统
子系统名称	交易系统管理业务
任务提出者	软件工程基础课程教学组
开发者	郝建文、黄中维、杨佳利、陆思越
用户群	股票交易所内部管理员（普通管理员、高级管理员、系统管理员、审计管理员）
外部交互系统	中央交易系统（TRADE）

2.2 项目背景

本项目为软件工程基础课程实验项目，属于“股票交易系统”五组协作项目中的交易系统管理业务子系统。五组分工如下：

- 1) 交易系统管理（**ADMIN** - 本组）：管理员登录、权限管理、股票查看、涨跌停设置、交易控制、交易日管理、审计日志。
- 2) 中央交易系统（**TRADE**）：指令接收与校验、订单簿维护、撮合引擎、成交回报、行情数据生成与推送。
- 3) 账户业务子系统（**ACCOUNT**）：资金账户与证券账户管理、认证、资金/证券冻结与结算。
- 4) 网上信息发布（**INFO**）：普通/VIP 用户体系、行情查询、K 线展示。
- 5) 交易客户端（**CLIENT**）：投资者侧交互界面，下单、撤单、查看持仓与成交。

本报告在需求分析报告与跨组接口约定的基础上进行系统设计。

2.3 需求概述

2.3.1 功能需求

本系统面向股票交易所内部管理员，采用基于角色的访问控制（RBAC），按职责划分为四类角色：

- 1) 普通管理员（**NORMAL_ADMIN**）：查看授权范围内的股票列表、实时行情与交易明细。
- 2) 高级管理员（**SENIOR_ADMIN**）：除普通管理员权限外，还具备涨跌停比例设置、交易暂停/重启、交易日开始/结束的控制权限。
- 3) 系统管理员（**SYSTEM_ADMIN**）：负责管理员账号的创建、角色分配、授权股票范围配置及账号状态管理（解锁、禁用、恢复启用）。
- 4) 审计管理员（**AUDIT_ADMIN**）：具有只读权限，查看全局操作日志与登录日志，支持按管理员、时间范围、操作类型筛选。

系统共划分为七个功能模块：登录管理、股票查看、涨跌停设置、交易控制（含交易日管理）、密码管理、权限管理、审计。

2.3.2 性能需求

- 1) 页面响应时间不超过 2 秒（100 并发下）；
- 2) 登录认证与权限识别响应时间小于 1 秒；
- 3) 查询类接口峰值 QPS 不低于 50；
- 4) 支持至少 100 个管理员并发访问。

2.3.3 安全性需求

- 1) 所有密码采用 bcrypt 加密存储，传输层使用 HTTPS；
- 2) 登录连续失败 5 次后账号临时锁定 5 分钟；
- 3) 会话超时自动退出，JWT 令牌含过期时间；
- 4) 所有关键操作记录操作日志，满足审计留痕要求；
- 5) 防止 SQL 注入、XSS、CSRF 等常见 Web 攻击。

2.4 条件与限制

2.4.1 技术约束

- 1) 后端统一采用 Python 3.11+ / FastAPI / Uvicorn 技术栈；
- 2) 数据库采用 MySQL 8.0+，ORM 选用 SQLAlchemy；
- 3) 接口风格遵循 RESTful，JSON 字段使用 snake_case，时间格式 ISO 8601；
- 4) 跨组通信通过 HTTP REST 接口，实时推送通过 WebSocket；
- 5) RabbitMQ 作为后续异步消息优化选项，第一版不要求引入。

2.4.2 外部依赖

- 1) 本子系统通过 HTTP 接口调用中央交易系统（TRADE）获取行情数据、订单簿数据，发送涨跌停配置、暂停/重启指令及交易日控制信号。ADMIN 子系统无需调用 ACCOUNT（管理员非投资者，无需资金/证券账户操作），仅通过 TRADE 的 API 与交易生态交互。第一版行情获取采用每 5 秒 REST 轮询。

2.4.3 约束限制

- 1) 本系统为课程实验项目，不涉及实际资金与证券交易；
- 2) 管理员的授权股票范围由系统管理员配置，普通管理员仅能查看授权范围内的数据；
- 3) 涨跌停比例设置后次日生效。

3 总体设计

3.1 基本设计概念和流程处理

3.1.1 设计概念

本系统采用前后端分离的 B/S (Browser/Server) 架构。后端遵循分层设计原则：

- 1) 路由层 (**Router**)：接收 HTTP 请求，参数校验，调用服务层。
- 2) 服务层 (**Service**)：实现核心业务逻辑，协调内部模块与外部 API 调用。
- 3) 数据访问层 (**Repository**)：封装数据库 CRUD 操作，通过 SQLAlchemy ORM 访问 MySQL。
- 4) 外部接口层 (**Client**)：封装对 TRADE 外部系统的 HTTP 调用。

前后端通过 RESTful API (JSON over HTTP) 通信，管理员认证采用 JWT Bearer Token。实时行情数据第一版采用每 5 秒 REST 轮询 TRADE /market 接口获取。

3.1.2 核心流程概述

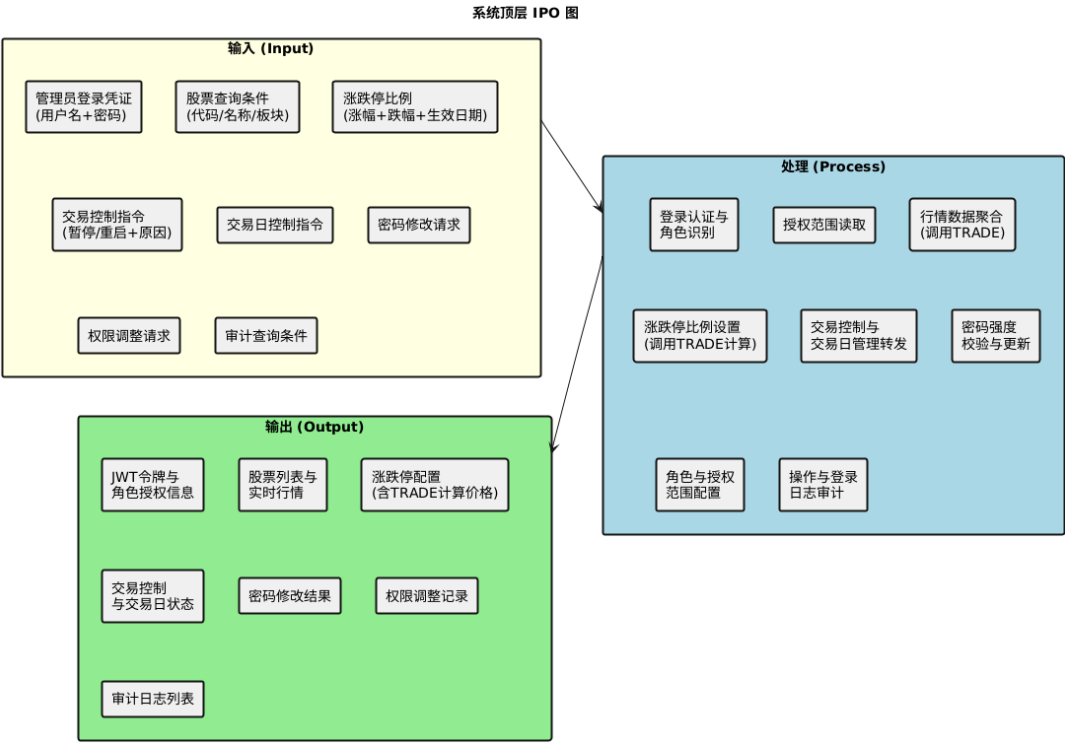
系统核心业务流程如下：

- 1) 管理员登录：管理员输入用户名与密码 → ADMIN 后端校验账号状态与密码 → 返回 JWT 令牌及角色与授权范围 → 前端根据角色展示对应功能菜单。
- 2) 股票查看：管理员进入股票查看界面 → ADMIN 后端从数据库读取授权范围 → 调用 TRADE /market 批量获取实时行情 → 前端展示。
- 3) 涨跌停设置：高级管理员提交涨跌停比例 → ADMIN 后端校验权限与授权范围 → 调用 TRADE PUT /stocks/{code}/limits 传递比例 → TRADE 计算最终价格限制并返回 → 管理员确认次日生效。
- 4) 交易控制：高级管理员发起暂停/重启 → ADMIN 后端校验权限 → 调用 TRADE 对应暂停/重启接口 → TRADE 执行操作并通过 WebSocket 广播状态变更 → ADMIN 记录审计日志。
- 5) 交易日管理：高级管理员在交易控制模块内发起交易日开始/结束指令 → ADMIN 后端调用 TRADE POST /trading-days/open 或 /close → TRADE 启动/停止撮合引擎、过期未成交指令、释放冻结资源、归档行情。

3.2 功能 IPO 图

系统的顶层 IPO 模型描述了主要的输入数据、核心处理功能和输出数据：

- 1) **Input** (输入)：管理员登录凭证 (用户名 + 密码)、股票查询条件 (代码/名称/板块)、涨跌停比例 (涨幅比例 + 跌幅比例 + 生效日期)、交易控制指令 (暂停/重启/交易日开始/交易日结束 + 原因)、密码修改请求、权限调整请求 (角色/授权范围/状态)、审计查询条件 (管理员/时间范围/操作类型)。
- 2) **Process** (处理)：登录认证与角色识别、授权范围读取、行情数据聚合 (调用 TRADE 接口)、涨跌停比例设置与转发 (调用 TRADE 计算)、交易控制与交易日管理指令转发与状态广播、密码强度校验与更新、角色与授权范围配置、操作与登录日志审计。
- 3) **Output** (输出)：认证结果与 JWT 令牌、股票列表与实时行情、涨跌停配置确认 (含 TRADE 计算的价格限制)、交易控制与交易日状态、密码修改结果、权限调整记录、审计日志列表。



3.3 系统结构

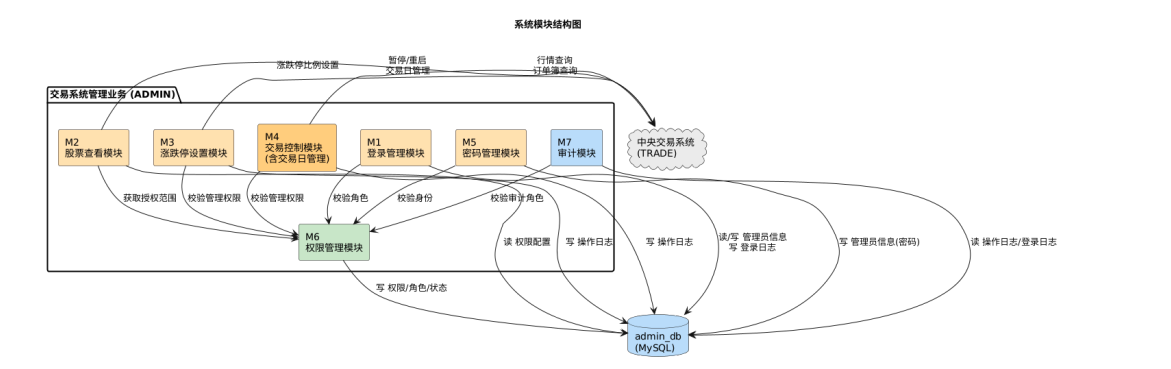
系统划分为以下七个功能模块：

编号	模块名称	功能简述	主要外部调用
M1	登录管理模块	管理员身份验证、会话创建与管理、登录失败锁定与自动解锁。	无（纯内部模块）
M2	股票查看模块	按授权范围展示股票列表、实时行情与交易明细，支持搜索筛选与排序。	TRADE: 行情查询、订单簿查询
M3	涨跌停设置模块	管理股票涨跌幅比例的单只与批量设置，调用 TRADE 完成价格计算，次日生效。	TRADE: 涨跌停比例设置
M4	交易控制模块	执行股票的暂停/重启撮合操作与交易日开始/结束，调用 TRADE 执行并触发广播。	TRADE: 暂停/重启、交易日管理
M5	密码管理模块	管理员密码修改、强度校验与修改成功后强制重新登录。	无（纯内部模块）
M6	权限管理模块	管理各管理员的角色分配、授权股票范围与账号状态配置。	无（纯内部模块）
M7	审计模块	提供全局操作日志与登录日志的查看、筛选与导出，满足合规留痕。	无（纯内部模块）

模块间依赖关系：

- 1) M2（股票查看）依赖 M6（权限管理）获取授权范围，依赖 TRADE 获取行情数据；
- 2) M3（涨跌停设置）依赖 M6（权限管理）校验管理权限，依赖 TRADE 计算价格限制；

- 3) M4（交易控制）依赖 M6（权限管理）校验管理权限，依赖 TRADE 执行暂停/重启及交易日启停；
- 4) M6（权限管理）仅系统管理员可访问；
- 5) M7（审计模块）仅审计管理员可访问。



3.4 技术介绍

本系统技术选型遵循五组统一约定（接口 V2.md 第 1.1 节），具体如下：

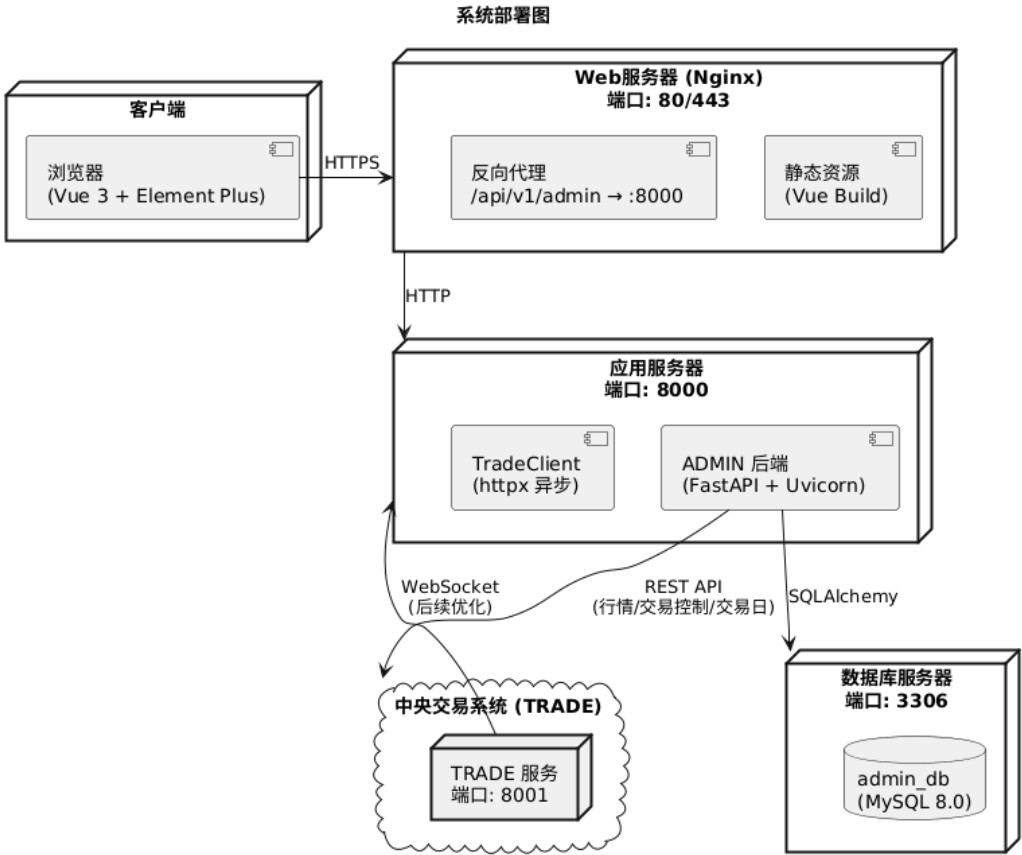
层次	技术/工具	选型理由
后端语言	Python 3.11+	五组统一约定，生态成熟，FastAPI 性能优秀
Web 框架	FastAPI	原生异步支持，自动生成 OpenAPI 文档，Pydantic 数据校验
ASGI 服务	Uvicorn	高性能 ASGI 服务器，支持 WebSocket
数据库	MySQL 8.0+	五组统一约定，支持事务与行级锁
ORM	SQLAlchemy 2.0+	异步支持，成熟的 Python ORM
数据库迁移	Alembic	与 SQLAlchemy 集成，版本化数据库变更
认证	python-jose (JWT)	无状态令牌，适合分布式部署
密码加密	bcrypt	抗暴力破解，业界标准
HTTP 客户端	httpx (异步)	用于调用 TRADE 外部 API
前端框架	Vue 3 + Element Plus	组件化开发，丰富的管理后台组件
版本控制	Git	分布式版本控制
绘图工具	draw.io / PlantUML	支持 UML 图与流程图

3.5 部署图

系统采用典型的 Web 应用部署架构。ADMIN 子系统部署在应用服务器上，通过 HTTP 协议与 TRADE 子系统通信，第一版采用 REST 轮询获取行情。

组件	部署位置	端口	说明
----	------	----	----

ADMIN 前 端 (Vue 3)	Nginx 静态资源 + 浏览器	80/443	Nginx 作为反向代理与静态 资源服务器
ADMIN 后 端 (FastAPI)	应用服务器	8000	处理业务逻辑，调用外部 API
MySQL (admin_db)	数据库服务器	3306	存储 管理员/日志/权限 数 据。第一版与 TRADE 等组 共用 MySQL 实例，通过库 名 admin_db 隔离
TRADE 服务	交易服务器	8001	外部依赖，通过 HTTP 交互 (第一版 REST 轮询行情)



3.6 类图

系统核心类设计如下，覆盖 MVC 各层：

3.6.1 实体类（Model）

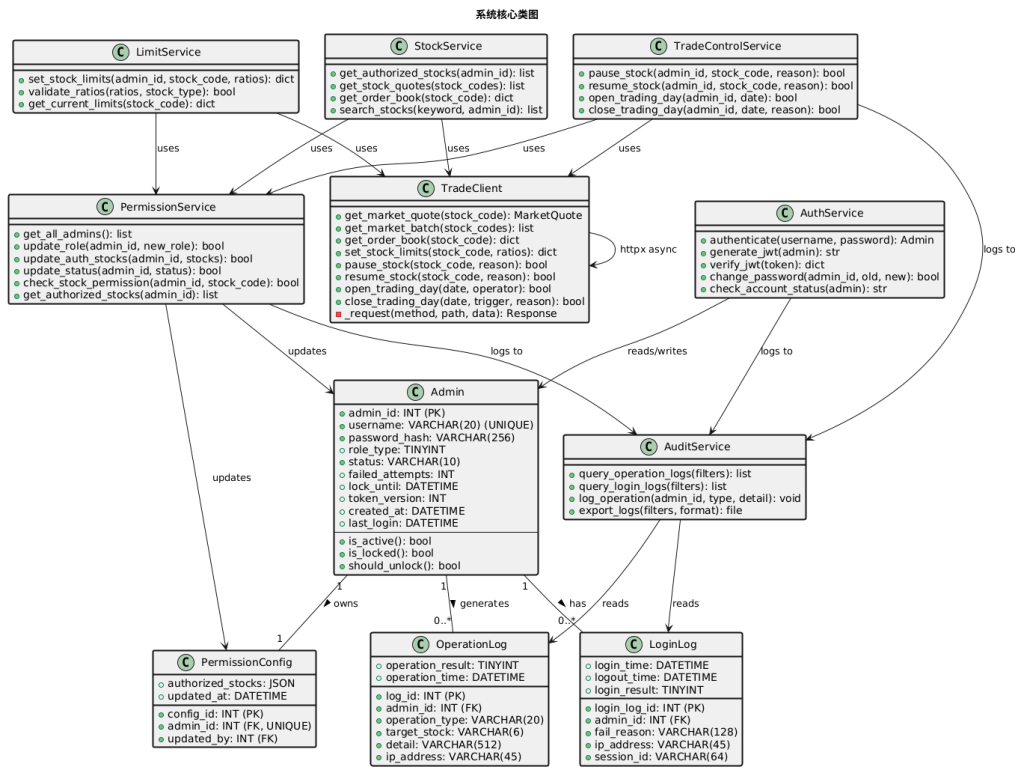
- 1) **Admin:** 管理员实体，属性包含 admin_id, username, password_hash, role_type, status, failed_attempts, lock_until, token_version, created_at, last_login。
- 2) **OperationLog:** 操作日志实体，属性包含 log_id, admin_id, operation_type, target_stock, detail, operation_result, operation_time, ip_address。
- 3) **PermissionConfig:** 权限配置实体，属性包含 config_id, admin_id, authorized_stocks (JSON), updated_by, updated_at。
- 4) **LoginLog:** 登录日志实体，属性包含 login_log_id, admin_id, login_time, logout_time, login_result, fail_reason, ip_address, session_id。

3.6.2 服务类（Service）

- 1) **AuthService**: 处理登录认证、JWT 令牌生成与校验、密码修改、会话管理。
- 2) **StockService**: 查询授权范围，调用 TRADE Client 获取行情与订单簿数据。
- 3) **LimitService**: 校验涨跌停比例合法性，调用 TRADE Client 设置比例。
- 4) **TradeControlService**: 调用 TRADE Client 执行暂停/重启/交易日管理，记录操作日志。
- 5) **PermissionService**: 管理员账号管理、角色调整、授权范围配置。
- 6) **AuditService**: 操作日志与登录日志的查询、筛选与导出。

3.6.3 外部接口类（Client）

- 1) **TradeClient**: 封装对 TRADE HTTPAPI 的调用（行情、订单簿、涨跌停、暂停/重启、交易日管理），第一版以 REST 轮询代替 WebSocket 连接。



3.7 接口设计

3.7.1 内部接口

内部接口采用 Service 层方法调用方式，各模块间通过明确的接口交互：

调用方	被调用方	接口描述
StockService	PermissionService	get_authorized_stocks(admin_id) → 获取管理员授权股票范围
LimitService	PermissionService	check_stock_permission(admin_id, stock_code) → 校验管理员对该股票的管理权限
TradeControlService	PermissionService	check_stock_permission(admin_id, stock_code) → 同上

AuthService	Admin (Model)	authenticate(username, password) → 验证身份并返回管理员信息
AuthService	AuditService	log_operation(...) → 记录操作日志
LimitService	TradeClient	set_stock_limits(stock_code, ...) → 向 TRADE 设置涨跌停比例
TradeControlService	TradeClient	pause_stock / resume_stock → 向 TRADE 发送控制指令
TradeControlService	TradeClient	open_trading_day / close_trading_day → 交易日管理
StockService	TradeClient	get_stock_quote(stock_code) → 从 TRADE 获取实时行情

3.7.2 外部接口（ADMIN 对外提供的 API）

ADMIN 子系统对外暴露以下 RESTful API（业务接口前缀 /api/v1/admin）：

方法	端点	描述
POST	/auth/login	管理员登录，返回 JWT 令牌、角色与授权范围
POST	/auth/logout	管理员退出登录，销毁会话
POST	/auth/password	修改当前管理员密码，成功后强制退出
GET	/stocks?keyword=	查询授权范围内的股票列表，支持关键字搜索
GET	/stocks/{stock_code}/quote	查询单只股票实时行情（从 TRADE 获取）
PUT	/stocks/{stock_code}/limits	设置涨跌停比例（转发至 TRADE 计算价格限制）
POST	/stocks/{stock_code}/pause	暂停指定股票交易
POST	/stocks/{stock_code}/resume	重启指定股票交易
POST	/trading-days/open	发起交易日开始（调用 TRADE）
POST	/trading-days/close	发起交易日结束（调用 TRADE）
GET	/admins	查看所有管理员账号与权限（仅系统管理员）
PUT	/admins/{admin_id}/permissions	调整管理员角色、授权范围或状态（仅系统管理员）
GET	/audit/operation-logs?filters...	查询操作日志（仅审计管理员），支持分页与筛选
GET	/audit/login-logs?filters...	查询登录日志（仅审计管理员），支持分页与筛选
GET	/audit/logs/export?filters...&format=csv	导出审计日志为 CSV 文件（仅审计管理员）
DELETE	/audit/logs?before=&type=	删除指定日期之前的操作日志或登录日志（仅审计管理员），需二次确认

以下端点不带 /api/v1/admin 前缀，直接挂在根路径：

方法	端点	描述
GET	/health	健康检查。检查 MySQL 数据库连通性（SELECT 1），检查 TRADE 服务 /health 可达性，汇总返回自身及各依赖状态。响应：{"service":"admin-service","status":"UP","dependencies":{"mysql":"UP","trade":"UP"},任一依赖不可用时 status="DEGRADED",自身不可用时返回 503。
GET	/docs	FastAPI 自动生成的 Swagger UI 交互式 API 文档
GET	/openapi.json	OpenAPI 3.0 JSON Schema，供其他组自动生成客户端代码

3.7.3 外部接口（ADMIN 调用的 TRADE API）

ADMIN 调用 TRADE 的接口（基础前缀 /api/v1/trade）：

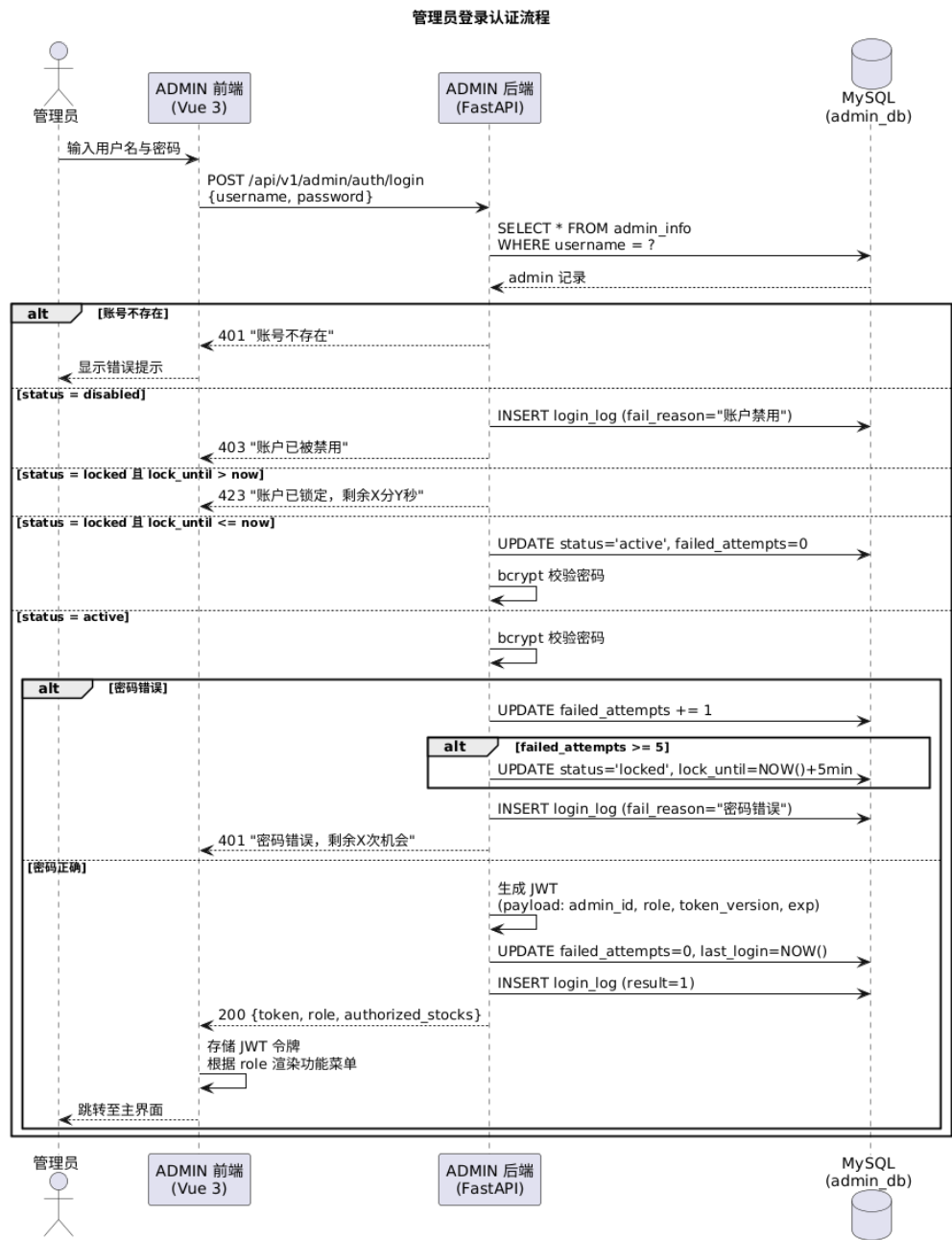
方法	端点	描述
GET	/market/{stock_code}	获取单只股票行情快照
GET	/market?stock_codes=	批量获取多只股票行情
GET	/order-books/{stock_code}	查询订单簿（买盘/卖盘档位）
PUT	/stocks/{stock_code}/limits	传递涨跌停比例，TRADE 计算并返回价格限制
POST	/stocks/{stock_code}/pause	暂停指定股票交易
POST	/stocks/{stock_code}/resume	重启指定股票交易
POST	/trading-days/open	交易日开始，启动撮合引擎
POST	/trading-days/close	交易日结束，停止撮合并过期未成交指令
GET	/orders/{order_id}	按需查询指令详情

4 详细设计

4.1 顺序图

顺序图描述各核心业务流程中 ADMIN 前端、ADMIN 后端、数据库及外部系统 (TRADE) 之间的消息交互序列。

4.1.1 管理员登录认证流程

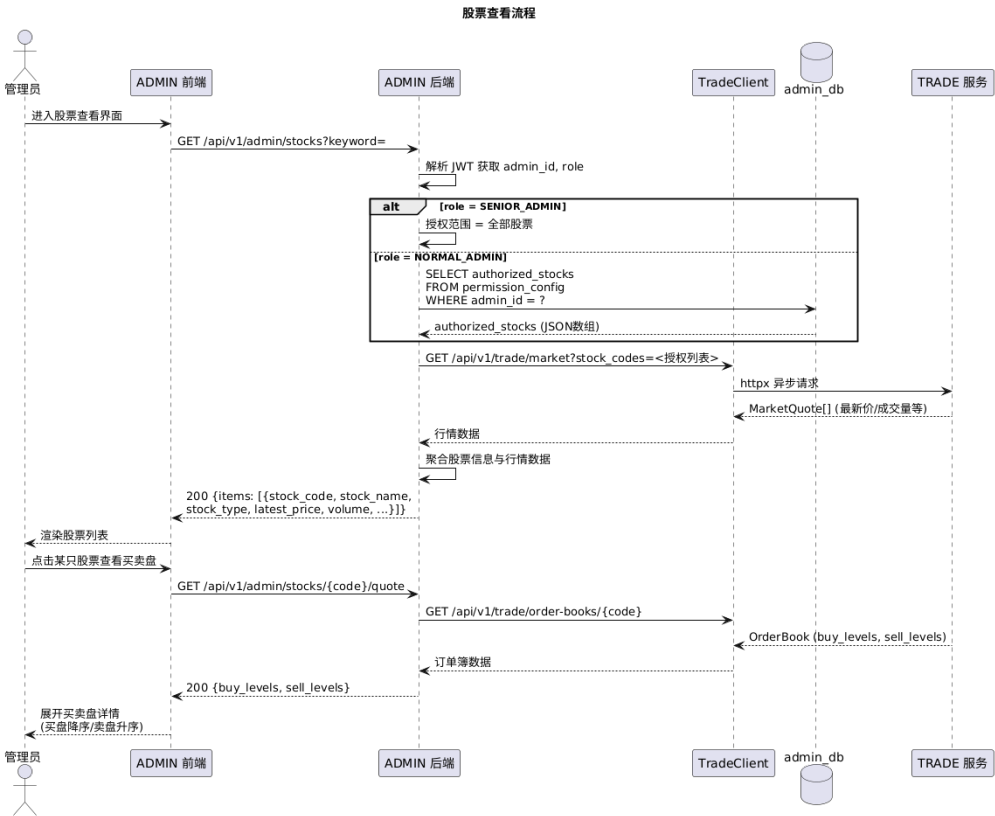


交互序列:

- 1) 管理员在浏览器输入用户名与密码, 提交 `POST /api/v1/admin/auth/login`;
- 2) ADMIN 后端查询 MySQL `admin_info` 表, 校验账号状态 (`active/locked/disabled`);
- 3) 若账号不可用, 返回 401/403 错误并记录登录失败日志;
- 4) 若账号可用, `bcrypt` 校验密码; 失败则累加 `failed_attempts`, 达到 5 次锁定 5 分钟;

- 5) 校验通过后生成 JWT 令牌（含 admin_id, role, 过期时间），清零失败计数，记录登录成功日志；
- 6) 返回令牌、角色（NORMAL_ADMIN / SENIOR_ADMIN / SYSTEM_ADMIN / AUDIT_ADMIN）与授权股票范围；
- 7) 前端存储 JWT 令牌，根据角色渲染对应的功能菜单。

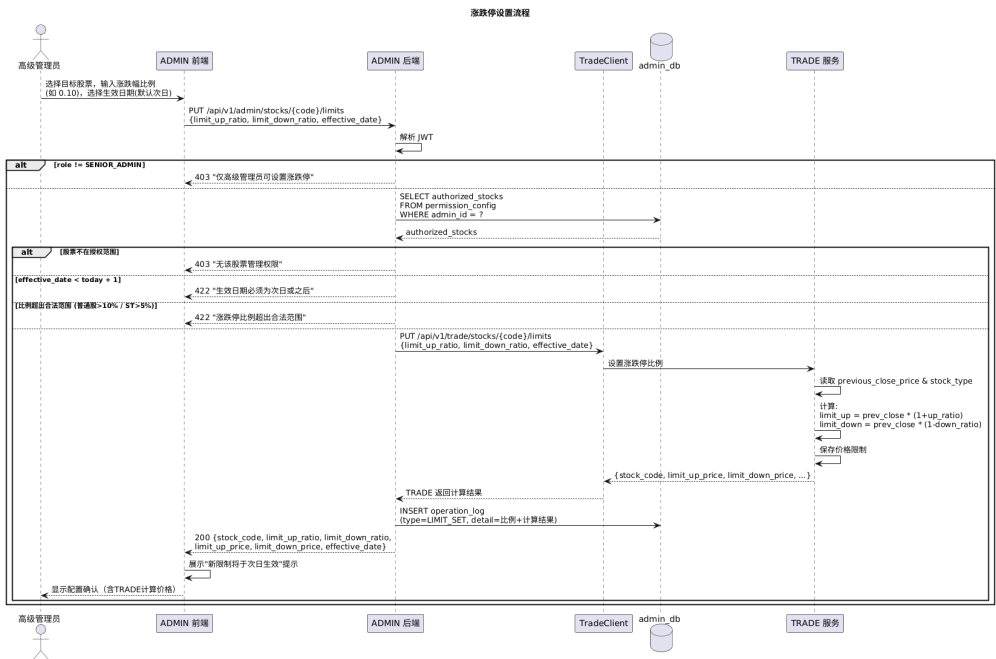
4.1.2 股票查看流程



交互序列：

- 1) 管理员进入股票查看界面，前端发送 GET /api/v1/admin/stocks?keyword=;
- 2) ADMIN 后端从 JWT 解析 admin_id，调用 PermissionService 查询该管理员的授权股票范围（permission_config 表）；
- 3) 若角色为高级管理员，授权范围为全部股票；若为普通管理员，从数据库读取 authorized_stocks 字段；
- 4) ADMIN 后端使用 httpx 异步调用 TRADE GET /api/v1/trade/market?stock_codes= 批量获取授权股票的行情快照；
- 5) TRADE 返回各股票的 MarketQuote（最新价、成交量、买卖盘等）；
- 6) ADMIN 后端聚合数据后返回前端渲染股票列表；
- 7) 管理员点击某只股票查看实时买卖盘，ADMIN 后端调用 TRADE GET /api/v1/trade/order-books/{stock_code} 获取订单簿数据。

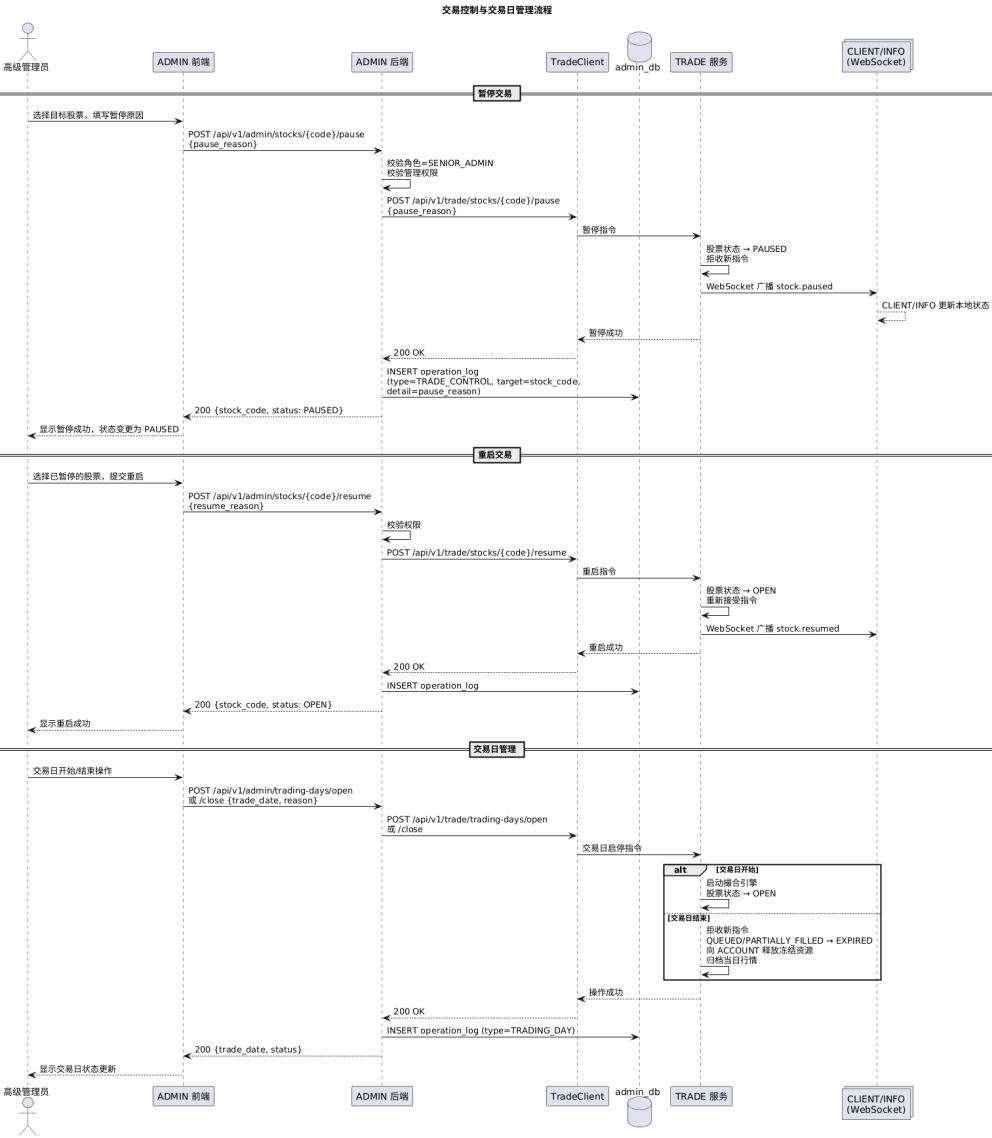
4.1.3 涨跌停设置流程



交互序列:

- 1) 高级管理员选择目标股票(支持多选), 输入涨跌幅比例, 提交 `PUT /api/v1/admin/stocks/{stock_code}/limits`;
- 2) ADMIN 后端从 JWT 校验角色必须为 SENIOR_ADMIN;
- 3) 调用 PermissionService 校验该管理员对该股票有管理权限;
- 4) 校验涨跌幅比例合法性 (普通股 $\leq 10\%$, ST 股 $\leq 5\%$);
- 5) ADMIN 后端调用 `TRADE PUT /api/v1/trade/stocks/{stock_code}/limits`, 传递比例参数;
- 6) TRADE 根据昨日收盘价、股票类型计算最终涨停价与跌停价, 保存并返回;
- 7) ADMIN 后端将 TRADE 返回的价格限制与配置信息返回前端展示;
- 8) 提示管理员“新限制将于次日(effective_date)生效”;
- 9) 向 operation_log 表写入涨跌停设置记录。

4.1.4 交易暂停与重启流程



交互序列（暂停）：

- 1) 高级管理员选择目标股票，填写暂停原因，提交 `POST /api/v1/admin/stocks/{stock_code}/pause`;
- 2) ADMIN 后端校验角色为 `SENIOR_ADMIN` 且对该股票有管理权限；
- 3) ADMIN 后端调用 `TRADE POST /api/v1/trade/stocks/{stock_code}/pause`，传递 `pause_reason`；
- 4) TRADE 将股票状态改为 `PAUSED`，拒收该股票的新指令，通过 `WebSocket` 广播 `stock.paused` 事件；
- 5) ADMIN 后端向 `operation_log` 表写入暂停操作记录（含暂停原因、操作时间、IP）；
- 6) 返回操作结果给前端。

交互序列（重启）：

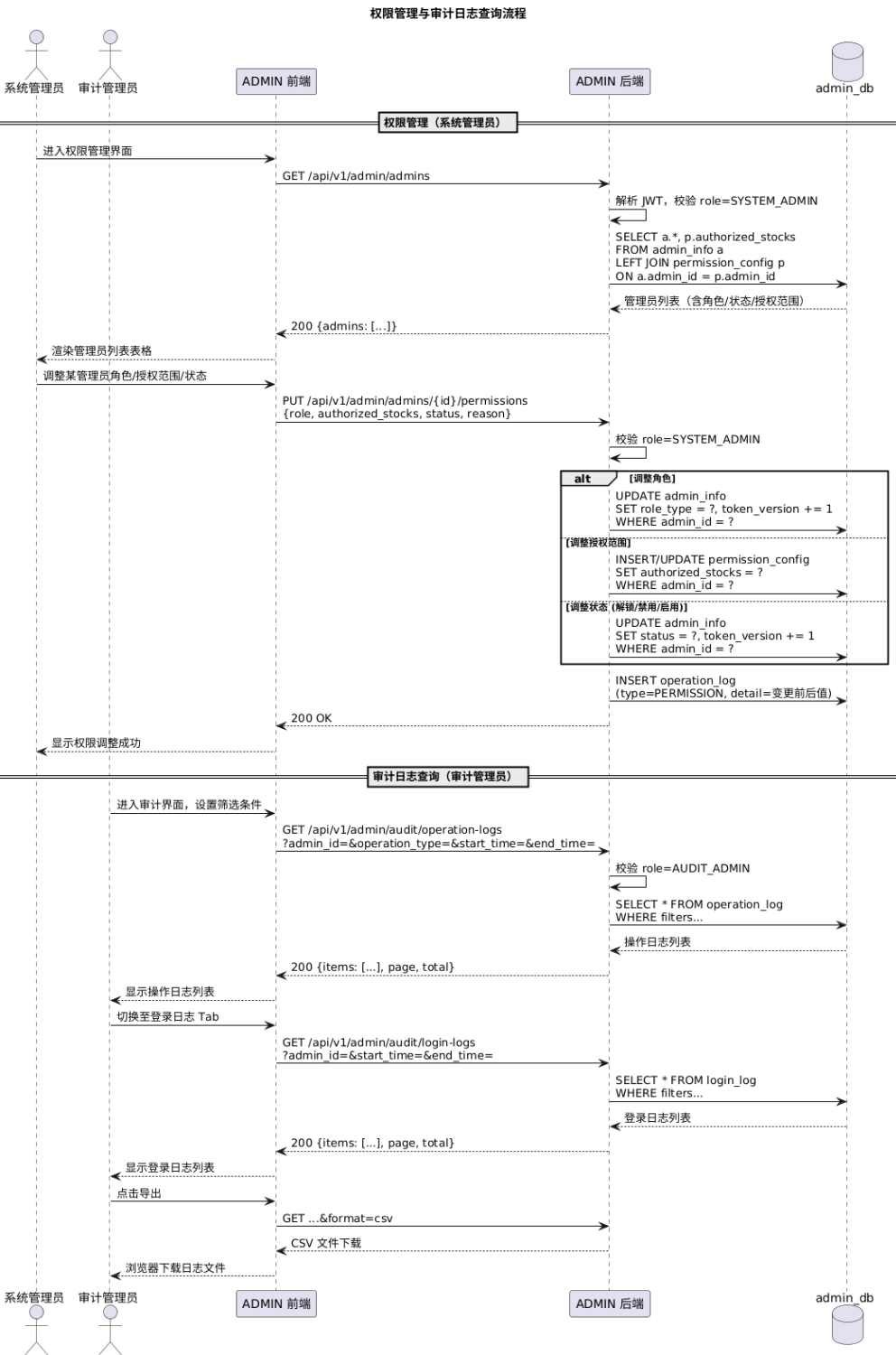
- 1) 高级管理员选择已暂停的股票，提交 `POST /api/v1/admin/stocks/{stock_code}/resume`；
- 2) ADMIN 后端校验权限后调用 `TRADE POST /api/v1/trade/stocks/{stock_code}/resume`；

- 3) TRADE 将股票状态恢复为 OPEN，通过 WebSocket 广播 stock.resumed 事件；
- 4) ADMIN 后端写入操作日志，返回操作结果。

交互序列（交易日管理）：

- 1) 交易日开始：高级管理员在交易日开始时（如周一至周五 9:00），提交 POST /api/v1/admin/trading-days/open；ADMIN 后端校验角色后调用 TRADE POST /api/v1/trade/trading-days/open；TRADE 启动撮合引擎；ADMIN 后端写入操作日志。
- 2) 交易日结束：高级管理员在交易日结束时（如 15:00）或系统定时任务触发，提交 POST /api/v1/admin/trading-days/close；ADMIN 后端调用 TRADE POST /api/v1/trade/trading-days/close；TRADE 停止撮合、过期未成交指令、释放冻结资源、归档行情；ADMIN 后端写入操作日志。

4.1.5 权限管理流程



交互序列:

- 1) 系统管理员进入权限管理界面, 前端发送 GET /api/v1/admin/admins;
- 2) ADMIN 后端校验角色为 SYSTEM_ADMIN;
- 3) 查询 admin_info 与 permission_config 联表, 返回所有管理员列表 (含角色、状态、授权范围);
- 4) 系统管理员调整某管理员的角色或授权范围, 提交 PUT /api/v1/admin/admins/{admin_id}/permissions;

- 5) ADMIN 后端更新 `admin_info` (角色/状态) 或 `permission_config` (授权范围) 表;
- 6) 向 `operation_log` 表写入权限变更记录 (含变更前后值)。

4.1.6 审计日志查询流程

交互序列:

- 1) 审计管理员进入审计界面, 前端可通过两个 Tab 切换操作日志与登录日志;
- 2) 操作日志: 前端发送 `GET /api/v1/admin/audit/operation-logs?filters...`; ADMIN 后端校验角色为 `AUDIT_ADMIN` (或 `SYSTEM_ADMIN`); 根据筛选条件查询 `operation_log` 表, 返回分页列表。
- 3) 登录日志: 前端发送 `GET /api/v1/admin/audit/login-logs?filters...`; ADMIN 后端按条件查询 `login_log` 表, 返回分页列表。
- 4) 支持导出日志文件 (CSV 格式)。

4.2 执行概念

4.2.1 登录认证执行逻辑

执行步骤:

- 1) 管理员输入用户名与密码, 提交登录请求;
- 2) 查询 `admin_info` 表获取账号记录, 若不存在则返回“账号不存在”;
- 3) 检查 `status` 字段: 若 `disabled`, 返回“账户已被禁用”; 若 `locked` 且未到 `lock_until`, 返回“账户已锁定, 剩余 X 分 Y 秒”; 若 `locked` 且已到解锁时间, 自动将 `status` 恢复为 `active`, `failed_attempts` 清零;
- 4) `bcrypt` 校验密码哈希: 成功则生成 JWT 令牌 (payload 含 `admin_id`, `role`, `exp`, `token_version`), 返回角色与授权范围, 记录登录成功日志; 失败则 `failed_attempts` += 1, 若达到 5 次则将 `status` 设为 `locked`, `lock_until` = now + 5 分钟, 返回错误提示并记录失败日志。

4.2.2 涨跌停设置执行逻辑

关键设计: ADMIN 只传递比例, TRADE 计算最终价格限制。

执行步骤:

- 1) 高级管理员在界面选择目标股票 (支持多只), 输入涨跌幅比例 (如 0.10 表示 ±10%) 和生效日期;
- 2) ADMIN 后端校验: 角色是否为 `SENIOR_ADMIN`、该管理员是否有对该股票的管理权限、比例是否在合法范围 (普通股 ≤ 0.10、ST 股 ≤ 0.05);
- 3) ADMIN 后端调用 `TRADE PUT /api/v1/trade/stocks/{stock_code}/limits`, 传递 `limit_up_ratio`, `limit_down_ratio`, `effective_date`;
- 4) TRADE 读取该股票的 `previous_close_price` (昨日收盘价) 和 `stock_type` (NORMAL/ST), 计算 `limit_up_price = previous_close_price * (1 + limit_up_ratio)`, `limit_down_price = previous_close_price * (1 - limit_down_ratio)`;
- 5) TRADE 保存价格限制并返回计算结果;
- 6) ADMIN 将生效日期、配置比例、TRADE 返回的价格限制一并展示给管理员确认;
- 7) 写入操作日志 (含股票代码、比例、生效日期、计算后的价格限制)。

4.2.3 交易控制执行逻辑

关键设计：ADMIN 发送控制信号（含交易日管理），TRADE 执行具体操作并通过 WebSocket 广播。

执行步骤（暂停/重启）：

- 1) 高级管理员选择目标股票（支持多只），若暂停需填写原因；
- 2) ADMIN 后端校验角色与权限；
- 3) 调用 TRADE POST /api/v1/trade/stocks/{stock_code}/pause（含 pause_reason）或 POST .../resume；
- 4) TRADE 内部：暂停时将该股票状态置为 PAUSED，订单簿拒收新指令；重启时恢复为 OPEN，重新接受指令；
- 5) TRADE 通过 WebSocket (/api/v1/trade/ws/market) 广播 stock.paused 或 stock.resumed 事件，CLIENT、INFO 等订阅方收到后更新本地状态；
- 6) ADMIN 后端写入操作日志（含操作类型、股票代码、原因、操作时间与 IP）。

执行步骤（交易日管理，属于交易控制模块的子功能）：

- 1) 开始：高级管理员在交易日开始时调用 POST /trading-days/open；ADMIN 后端调用 TRADE 同名接口；TRADE 启动撮合引擎，将所有股票状态初始化为 OPEN。
- 2) 结束：高级管理员手动或系统定时任务触发 POST /trading-days/close；TRADE 停止接收新指令，将所有 QUEUED 和 PARTIALLY_FILLED 指令标记为 EXPIRED，生成过期反馈，向 ACCOUNT 发送释放剩余冻结资源请求，归档当日行情数据。

4.2.4 密码管理执行逻辑

- 1) 管理员输入原密码、新密码及确认密码；
- 2) ADMIN 后端校验原密码正确性，校验新密码格式（长度 ≥ 8 、含大小写字母/数字/特殊字符至少三类）且两次输入一致；
- 3) 更新 admin_info 表中 password_hash 字段，同时递增 token_version 字段使所有旧 JWT 失效；
- 4) 强制前端清除令牌并跳转至登录界面；
- 5) 写入操作日志。

4.2.5 权限管理执行逻辑

- 1) 系统管理员查看所有管理员列表；
- 2) 可调整目标管理员的角色（NORMAL_ADMIN $\leftrightarrow$ SENIOR_ADMIN $\leftrightarrow$ AUDIT_ADMIN）、授权股票范围(JSON 数组)、账号状态(active/locked/disabled)；
- 3) 更新 admin_info 或 permission_config 表；
- 4) 写入操作日志（含目标管理员、变更前后值）。

5 用户界面

5.1 设计原则

- 1) 一致性：统一的 Element Plus 组件风格、色彩方案（主色调蓝色）与操作逻辑；
- 2) 角色适配：根据管理员角色动态展示功能菜单，无权限的功能入口隐藏或置灰；
- 3) 清晰性：关键数据（股票行情、涨跌停配置）突出显示，操作按钮位置统一；
- 4) 反馈性：操作结果使用 ElMessage 即时反馈（成功 / 失败 / 加载中），危险操作提供二次确认弹窗；
- 5) 响应式：适配 1366×768 至 1920×1080 主流分辨率。

5.2 界面原型

5.2.1 登录界面

- 1) 居中登录卡片，包含用户名输入框与密码输入框；
- 2) 登录按钮（验证码作为后续 bonus 功能实现）；
- 3) 登录失败时显示错误提示（用户名或密码错误 / 账户已锁定 X 分 Y 秒 / 账户已禁用）。

5.2.2 股票查看主界面

- 1) 顶部：导航栏（系统名称、当前管理员信息、退出按钮）；
- 2) 左侧：功能菜单（股票查看、涨跌停设置、交易控制、密码修改、权限管理、审计日志——根据角色显示）；
- 3) 主区域：股票列表（表格：代码、名称、类型、最新价、涨跌幅、成交量、状态），支持关键字搜索与板块筛选；
- 4) 点击某只股票：展开买卖盘详情（买盘降序、卖盘升序），每档显示价格、数量、时间。

5.2.3 涨跌停设置界面

- 1) 股票选择区：下拉多选或搜索选择授权范围内的股票；
- 2) 配置区：涨幅比例输入框（如 0.10）、跌幅比例输入框（如 0.10）、生效日期选择器（默认次日）；
- 3) 预览区：显示 TRADE 返回的昨日收盘价与计算后的涨停价、跌停价；
- 4) 提交按钮与重置按钮。

5.2.4 交易控制界面

交易控制界面包含两个操作区（通过 Tab 切换）：

交易暂停/重启区：

- 1) 股票选择区：下拉选择可管理的股票；
- 2) 操作区：暂停按钮（点击后弹出暂停原因输入弹窗）、重启按钮（已暂停的股票才可用）；
- 3) 状态展示：当前股票的交易状态（OPEN / PAUSED / CLOSED），使用不同颜色标识。

交易日管理区（仅 SENIOR_ADMIN 可见）：

- 1) 当前交易日状态显示（已开始/已结束）；
- 2) 交易日开始按钮（仅在未开始时可用）；
- 3) 交易日结束按钮（仅在已开始时可用，点击前二次确认）。

5.2.5 权限管理界面

- 1) 管理员列表表格（用户名、角色、状态、授权股票数、最近登录时间）；
- 2) 点击某管理员：弹出编辑面板（调整角色下拉框、多选股票选择器、状态切换）；
- 3) 系统管理员仅可管理非自身的其他管理员。

5.2.6 审计日志界面

- 1) 筛选区：按管理员下拉选择、按操作类型下拉选择、按时间范围日期选择器；
- 2) 日志列表表格（时间、操作管理员、操作类型、目标对象、详情、结果、IP）；
- 3) 导出按钮（导出当前筛选结果为 CSV 文件）。

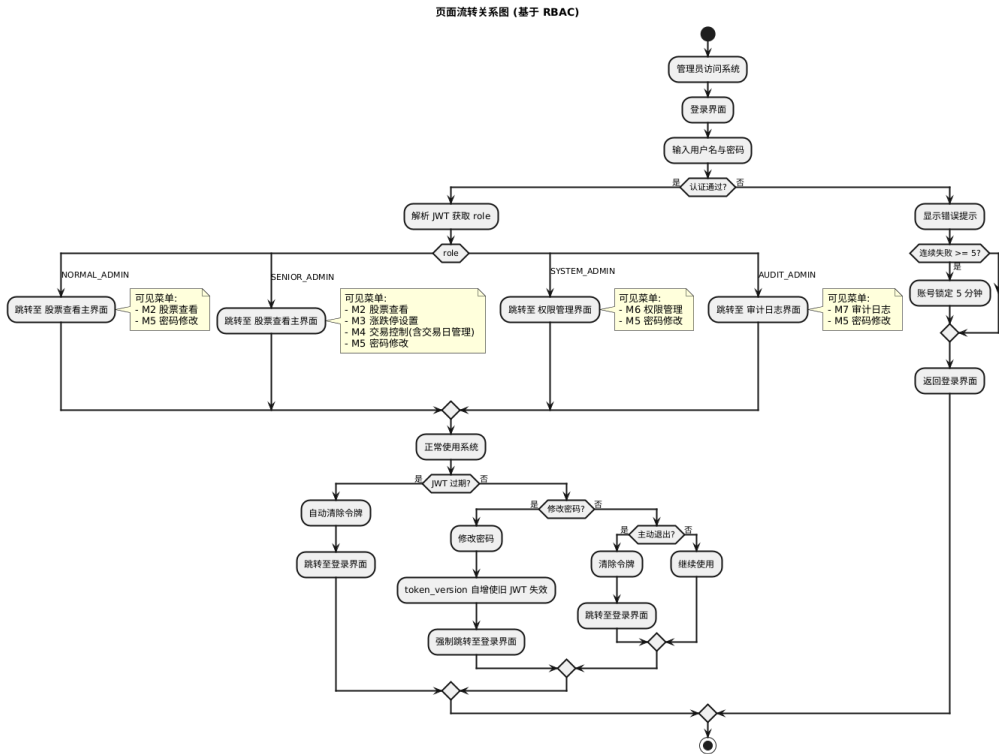
5.2.7 密码修改界面

- 1) 原密码输入框、新密码输入框、确认密码输入框；
- 2) 新密码强度实时指示器（弱/中/强）；
- 3) 提交按钮与取消按钮。

5.3 页面流转关系

页面流转规则（基于 RBAC）：

- 1) 所有管理员从登录页进入，登录成功后根据 role 字段跳转：
 - NORMAL_ADMIN → 股票查看主界面（仅可见 M2, M5 菜单项）；
 - SENIOR_ADMIN → 股票查看主界面（可见 M2, M3, M4（含交易日管理）, M5 菜单项）；
 - SYSTEM_ADMIN → 权限管理界面（可见 M6, M5 菜单项）；
 - AUDIT_ADMIN → 审计日志界面（可见 M7, M5 菜单项）；
- 2) 所有角色均可通过导航栏右上角进入密码修改（M5）；
- 3) 会话超时（JWT 过期）后前端自动清除令牌并跳转至登录界面；
- 4) 密码修改成功后（token_version 自增使所有旧令牌失效）强制跳转至登录界面。



6 数据库设计

6.1 概念结构设计

6.1.1 实体描述

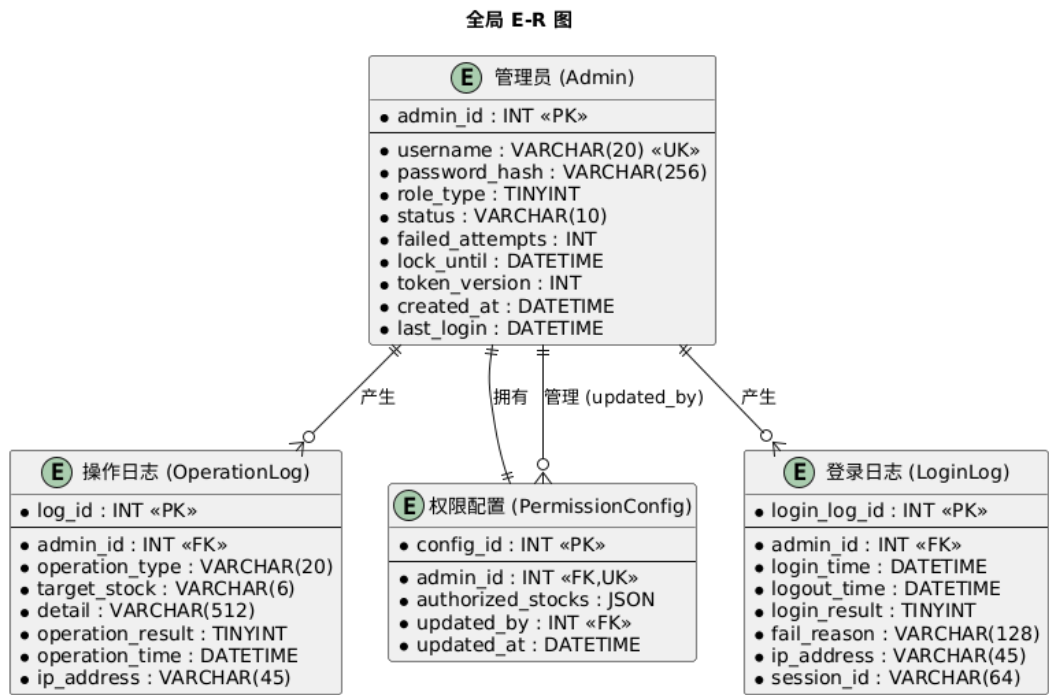
ADMIN 子系统独立维护以下四个实体（仅存储本子系统所需数据，股票行情、交易指令等数据通过 TRADE API 获取）：

- 1) 管理员（**Admin**）：属性包括管理员 ID、用户名、密码哈希、角色类型（NORMAL_ADMIN / SENIOR_ADMIN / SYSTEM_ADMIN / AUDIT_ADMIN）、账号状态（active / locked / disabled）、失败计数、锁定截止时间、令牌版本号（token_version，密码修改/禁用时自增）、创建时间、最近登录时间。
- 2) 操作日志（**OperationLog**）：属性包括日志 ID、管理员 ID、操作类型（LOGIN / QUERY / LIMIT_SET / TRADE_CONTROL / TRADING_DAY / PASSWORD / PERMISSION / ADMIN_STATUS）、目标股票代码、操作详情、操作结果、操作时间、IP 地址。
- 3) 权限配置（**PermissionConfig**）：属性包括配置 ID、管理员 ID、授权股票代码列表（JSON 数组）、最近修改者 ID、最近修改时间。
- 4) 登录日志（**LoginLog**）：属性包括日志 ID、管理员 ID、登录时间、登出时间、登录结果、失败原因、IP 地址、会话 ID。

6.1.2 实体间关系

- 1) 管理员（Admin）与 操作日志（OperationLog）：一对多 —— 一个管理员可产生多条操作日志记录。
- 2) 管理员（Admin）与 登录日志（LoginLog）：一对多 —— 一个管理员可产生多条登录日志记录。
- 3) 管理员（Admin）与 权限配置（PermissionConfig）：一对一 —— 每个管理员有一份权限配置（含授权股票范围）。

6.1.3 E-R 图



6.2 逻辑结构设计

数据库名称: admin_db, 字符集: utf8mb4, 排序规则: utf8mb4_unicode_ci。

6.2.1 管理员信息表 (admin_info)

序号	字段名	类型	长度	主键	说明
1	admin_id	INT	8	是	管理员唯一标识, 自增主键
2	username	VARCHAR	20		登录用户名, 唯一索引 UNIQUE
3	password_hash	VARCHAR	256		密码哈希值, bcrypt 加密存储
4	role_type	TINYINT	1		角色 : 0=NORMAL_ADMIN, 1=SENIOR_ADMIN, 2=SYSTEM_ADMIN, 3=AUDIT_ADMIN
5	status	VARCHAR	10		账号状态: active / locked / disabled
6	failed_attempts	INT	1		连续登录失败次数, 默认 0, 达到 5 触发锁定
7	lock_until	DATETIME			锁定截止时间, NULL 表示未锁定
8	token_version	INT	4		令牌版本号, 默认 1; 密码修改/账号禁用时自增使所有旧 JWT 失效
9	created_at	DATETIME			账号创建时间, 默认 CURRENT_TIMESTAMP
10	last_login	DATETIME			最近一次登录成功时间

6.2.2 操作日志表 (operation_log)

序号	字段名	类型	长度	主键	说明
1	log_id	INT	8	是	日志唯一标识, 自增主键
2	admin_id	INT	8		操作管理员 ID, 外键 REFERENCES admin_info(admin_id)
3	operation_type	VARCHAR	20		操作类型: LOGIN / QUERY / LIMIT_SET / TRADE_CONTROL / TRADING_DAY / PASSWORD / PERMISSION / ADMIN_STATUS
4	target_stock	VARCHAR	6		操作目标股票代码, 非股票操作时空
5	detail	VARCHAR	512		操作详细描述(含变更前后值、原因等)
6	operation_result	TINYINT	1		操作结果: 0=失败, 1=成功
7	operation_time	DATETIME			操作时间, 默认 CURRENT_TIMESTAMP, 精确到秒
8	ip_address	VARCHAR	45		操作来源 IP 地址, 支持 IPv6

6.2.3 权限配置表 (permission_config)

序号	字段名	类型	长度	主键	说明
1	config_id	INT	8	是	配置唯一标识, 自增主键
2	admin_id	INT	8		管理员 ID, 外键 REFERENCES admin_info(admin_id), UNIQUE
3	authorized_stocks	JSON			授权股票代码列表, 如 ["600000", "000001"], MySQL 8.0 JSON 类型
4	updated_by	INT	8		最近修改者 (系统管理员 ID), 外键 REFERENCES admin_info(admin_id)
5	updated_at	DATETIME			最近修改时间, 更新时自动设置

6.2.4 登录日志表 (login_log)

序号	字段名	类型	长度	主键	说明
1	login_log_id	INT	8	是	登录日志唯一标识, 自增主键
2	admin_id	INT	8		管理员 ID, 外键 REFERENCES admin_info(admin_id)

3	login_time	DATETIME			登录请求时间
4	logout_time	DATETIME			登出时间，NULL 表示会话尚未结束
5	login_result	TINYINT	1		登录结果：0=失败，1=成功
6	fail_reason	VARCHAR	128		登录失败原因(如“密码错误”/“账户锁定”/“账户禁用”)，成功时为 空
7	ip_address	VARCHAR	45		登录来源 IP 地址
8	session_id	VARCHAR	64		会话标识 (JWT jti)，登出后置为 NULL

所 有 外 键 （ operation_log.admin_id、 login_log.admin_id、 permission_config.admin_id、 permission_config.updated_by）均设置为 ON DELETE RESTRICT，禁止物理删除管理员账号。业务上仅将管理员 status 设为 disabled 以实现停用，不执行 DELETE 操作。

6.3 物理结构设计

6.3.1 存储引擎

所有表使用 InnoDB 存储引擎，支持事务（ACID）、行级锁与外键约束，确保操作日志写入与管理操作的数据一致性。

6.3.2 索引设计

表名	索引字段	索引类型	说明
admin_info	username	UNIQUE	用户名唯一，防止重复创建
admin_info	role_type	INDEX	按角色查询管理员列表
admin_info	status	INDEX	快速筛选锁定/禁用账号
operation_log	admin_id	INDEX	按操作管理员查询日志
operation_log	operation_type	INDEX	按操作类型筛选
operation_log	operation_time	INDEX	按时间范围筛选审计查询
login_log	admin_id	INDEX	按管理员查询登录记录
login_log	login_time	INDEX	按时间范围筛选
permission_config	admin_id	UNIQUE	每管理员唯一一份权限配置

6.3.3 备份策略

- 1) 每日全量备份：通过 MySQL mysqldump 定时任务每日凌晨全量导出 admin_db，保留最近 7 天；
- 2) 二进制日志（binlog）：开启 binlog 实现时间点恢复（PITR），日志保留 7 天；
- 3) 从库复制：建议配置 MySQL 主从复制，从库用于读写分离与灾难恢复（可选）。

7 运行设计

7.1 运行环境

7.1.1 服务器环境

项目	配置要求
CPU	4 核 2.6GHz 或以上
内存	8.0 GB 或以上
硬盘	7200 转，可用空间不小于 50GB（含日志存储）
操作系统	Ubuntu 20.04 LTS / 22.04 LTS 或 CentOS 7/8
Python	3.11+
数据库	MySQL 8.0.32+
Web 服务器	Nginx 1.20+（反向代理 + 静态资源）
网络	千兆以太网，与 TRADE 子系统内网互通

7.1.2 客户端环境

- 1) 操作系统：Windows 10/11、macOS、Linux；
- 2) 浏览器：Chrome（最新 2 个版本）、Edge（最新 2 个版本）、Firefox（最新版）；
- 3) 屏幕分辨率：不小于 1366×768。

7.2 运行流程

7.2.1 系统启动

- 1) 启动 MySQL 数据库服务，确认 admin_db 数据库可连接；
- 2) 启动 ADMIN 后端服务：uvicorn app.main:app --host 0.0.0.0 --port 8000；
- 3) 启动 Nginx：加载配置，反向代理 /api/v1/admin 到后端，静态资源直接由 Nginx 提供；
- 4) 确认 TRADE 服务已启动且 /health 可访问；
- 5) 系统管理员登录后可开始日常操作。

7.2.2 系统运行

- 1) 管理员登录后根据角色进入对应功能界面；
- 2) ADMIN 后端处理管理员请求：内部模块（登录、密码、权限、审计）直接操作数据库；外部依赖（行情、涨跌停、交易控制、交易日管理）通过 httpx 异步调用 TRADE API；
- 3) 实时行情数据通过每 5 秒 REST 轮询 TRADE /market 获取并缓存；
- 4) 所有关键操作写入 operation_log 表。

7.2.3 系统维护

- 1) 定期检查 operation_log 与 login_log 表大小，必要时导出旧日志文件并清理记录，防止表过度膨胀；
- 2) 定期执行数据库备份；
- 3) 监控系统性能（CPU、内存、磁盘、请求响应时间、TRADE 调用延迟）；
- 4) 按需更新 Python 依赖与系统安全补丁。

7.2.4 系统关闭

- 1) 高级管理员（或系统定时任务）调用交易日结束接口；
- 2) ADMIN 后端停止接收新的管理请求，等待正在处理的请求完成；
- 3) 依次关闭 Uvicorn、Nginx、MySQL。

7.3 运行控制

- 1) Nginx 配置请求限流（`limit_req_zone`），防止暴力登录攻击；
- 2) 应用层通过 FastAPI 中间件实现：JWT 认证校验、角色权限校验、请求日志记录、全局异常捕获与统一错误响应；
- 3) 数据库连接池管理（`SQLAlchemy pool_size=10,max_overflow=20`），防止连接泄漏；
- 4) 操作日志异步写入，避免阻塞主业务流程；
- 5) 外部 TRADE API 调用设置超时（连接超时 3s，读取超时 5s），失败时重试 1 次并记录错误日志。

8 系统出错设计

8.1 出错信息

系统通过统一的错误响应格式返回错误信息（false 时携带 code 与 message）。各错误码含义如下：

- 1) **400 COMMON_BAD_REQUEST** — 请求参数错误：请求体字段缺失或格式不正确，前端表单校验应拦截大部分此类错误。
- 2) **401 COMMON_UNAUTHORIZED** — 未认证：JWT 令牌缺失、无效或已过期，前端跳转至登录页。
- 3) **403 COMMON_FORBIDDEN** — 无权限：当前角色无权执行该操作（如普通管理员尝试设置涨跌停）。
- 4) **404 COMMON_NOT_FOUND** — 资源不存在：请求的股票代码、管理员 ID 等不存在。
- 5) **409 COMMON_CONFLICT** — 状态冲突：如对已暂停的股票再次暂停、对未暂停的股票重启。
- 6) **422 ADMIN_VALIDATION_ERROR** — 数据校验失败：密码强度不足、涨跌停比例超出合法范围、生效日期早于当日。
- 7) **429 ADMIN_RATE_LIMITED** — 请求过于频繁：登录失败次数过多被临时锁定，或 API 调用超过限流阈值。
- 8) **500 COMMON_INTERNAL_ERROR** — 服务器内部错误：未预期的运行时异常，后端记录完整 traceback，返回通用错误提示。
- 9) **502 ADMIN_UPSTREAM_ERROR** — 上游服务错误：调用 TRADE 接口失败（超时/5xx），返回“外部服务暂不可用”。
- 10) **503 COMMON_SERVICE_UNAVAILABLE** — 服务不可用：系统维护中或数据库连接失败。

8.2 补救措施

8.2.1 数据备份与恢复

- 1) 数据库每日全量备份（mysqldump），保留 7 天；
- 2) 开启 MySQL binlog，支持任意时间点恢复；
- 3) 定期演练数据恢复流程，验证备份文件可用性。

8.2.2 故障恢复

- 1) **Web 服务器（Nginx）故障**：重启 Nginx，若不可恢复则切换至备用节点；
- 2) **ADMIN 后端故障**：Uvicorn 配置 --workers 4 多进程，单进程崩溃后自动重启；若全部不可用，运维手动重启服务；
- 3) **MySQL 故障**：若主库故障，切换至从库并提升为主库；若无从库，从最近备份恢复；
- 4) **TRADE 不可用**：ADMIN 后端缓存最近一次成功获取的行情数据（5 秒过期），向前端展示“行情数据延迟”提示，交易控制类操作返回 502 错误并提示稍后重试。

8.2.3 安全事件响应

- 1) **异常登录检测**：同一 IP 短时间内多次登录失败，Nginx 限流 + 应用层拒绝；
- 2) **账号被盗**：审计管理员发现异常操作日志后，通知系统管理员立即禁用相关账号；
- 3) **漏洞攻击**：记录攻击特征到日志，通过 WAF（Web 应用防火墙）规则或 Nginx 规则封禁攻击 IP，及时修复漏洞；

4) 数据泄露：操作日志定期审计，异常查询行为追溯。

8.3 系统维护设计

- 1) 版本控制：所有代码通过 Git 管理，提交前通过代码审查；发布分支与开发分支分离。
- 2) 依赖管理：使用 `requirements.txt` 或 `pyproject.toml` 锁定依赖版本，定期运行 `pip audit` 检查安全漏洞。
- 3) 日志维护：`operation_log` 与 `login_log` 表不设自动归档，通过审计模块提供的导出（CSV）与删除功能手动清理。导出后由运维人员决定保留策略。
- 4) 性能监控：通过 FastAPI 内置 `metrics` + 中间件记录请求耗时，监控 TRADE API 调用延迟与成功率。
- 5) 接口文档：通过 FastAPI 自动生成 `/docs`（Swagger UI）和 `/openapi.json`，与跨组接口约定保持同步。